

Documentation Technique

Implémentation de l'algorithme RSA
et de la Signature Numérique

Projet de démonstration cryptographique en Python

Réalisé par :BELKACEM Kenza Étudiante en Ingénierie de
l'Intelligence Artificielle
2025/2026

1. Introduction et Contexte

Le projet présenté ici est une implémentation logicielle en Python du cryptosystème **RSA** (Rivest-Shamir-Adleman). Développé à des fins purement pédagogiques, ce programme permet de manipuler les concepts fondamentaux de la cryptographie asymétrique à travers une interface en ligne de commande (CLI).

Le programme illustre quatre opérations cryptographiques de base :

1. Le chiffrement d'un message clair.
2. Le déchiffrement d'un cryptogramme.
3. La génération d'une signature numérique.
4. La vérification de l'authenticité d'une signature.

L'objectif de ce document est de détailler les fondements mathématiques utilisés, l'architecture algorithmique du script, et de fournir une analyse critique de cette implémentation face aux standards de sécurité modernes.

2. Fondements Mathématiques

La sécurité de RSA repose sur la difficulté algorithmique de factoriser de grands nombres premiers (problème de la factorisation entière).

2.1 Génération des clés

- Soient p et q , deux grands nombres premiers distincts.
- Le module n est calculé par : $n = p \times q$.
- L'indicatrice d'Euler est définie par : $\phi(n) = (p - 1)(q - 1)$.
- L'exposant public e est choisi tel que : $1 < e < \phi(n)$ et $\text{PGCD}(e, \phi(n)) = 1$.
- L'exposant privé d est l'inverse modulaire de e modulo $\phi(n)$, vérifiant : $e \times d \equiv 1 \pmod{\phi(n)}$.

Exemple dans le code : $n = 3233$ (61×53), $e = 17$, $d = 2753$.

2.2 Chiffrement et Déchiffrement

Pour un bloc de message M (avec $M < n$) :

- **Chiffrement** : $C \equiv M^e \pmod{n}$
- **Déchiffrement** : $M \equiv C^d \pmod{n}$

2.3 Signature et Vérification

La signature garantit l'intégrité et l'authenticité (non-répudiation). On utilise une fonction de hachage H (ici SHA-256).

- **Signature** : $S \equiv H(M)^d \pmod{n}$
- **Vérification** : Calculer $H(M) \pmod{n}$ et $S^e \pmod{n}$. Si les deux valeurs sont identiques, la signature est valide.

3. Architecture et Conception du Logiciel

Le code est structuré de manière modulaire autour de fonctions pures, séparant la logique mathématique de l'interface utilisateur.

3.1 Exponentiation Modulaire

La fonction `mod_exp(base, exp, mod)` s'appuie sur la primitive native de Python `pow(base, exp, mod)`. En Python 3, cette primitive utilise l'algorithme d'exponentiation modulaire binaire (square-and-multiply), garantissant une complexité temporelle en $O(\log e)$ et une protection native contre les attaques par canaux auxiliaires (timing attacks).

3.2 Gestion des Blocs (Texte vers Entier)

La fonction `texte_vers_blocs` convertit le message en une liste d'entiers représentant les codes ASCII de chaque caractère.

- **Contrainte** : Une vérification assure que $\text{code} < n$. Si n est trop petit, le programme lève une erreur, évitant ainsi un modulo involontaire qui rendrait le déchiffrement impossible.

3.3 Fonction de Hachage

La fonction `H(message)` utilise la librairie standard `hashlib` pour générer une empreinte SHA-256. Le résultat hexadécimal est converti en un entier décimal très grand, servant de base à la signature numérique.

4. Analyse Critique et Limites de Sécurité

Bien que fonctionnellement correct selon les formules mathématiques du RSA "textbook" (RSA brut), ce programme présente des caractéristiques qui le rendent **inapte à un usage en production**. Cette section démontre la compréhension des enjeux de sécurité réels.

4.1 Faiblesse du Chiffrement Caractère par Caractère

L'implémentation actuelle chiffre chaque caractère indépendamment ($M < 256$). Si le texte contient des répétitions (ex : le lettre 'e'), les cryptogrammes C seront identiques. Cette approche est vulnérable à une **analyse fréquentielle** simple, annihilant la sécurité de l'algorithme. *Standard moderne* : Utiliser un schéma de remplissage (padding) comme **OAEP** (Optimal Asymmetric Encryption Padding) qui regroupe les caractères en blocs de la taille de n et y ajoute de l'aléatoire.

4.2 Taille des Clés

L'exemple utilise $n = 3233$ (codé sur 12 bits). Un tel module peut être factorisé en une fraction de seconde par n'importe quel ordinateur moderne. *Standard moderne* : L'ANSSI (Agence Nationale de la Sécurité des Systèmes d'Information) recommande une taille de clé minimale de 2048 bits pour un usage standard, et 3072 bits pour une durée de validité supérieure à 2030.

4.3 Signature sans Padding (PSS)

La vérification de la signature applique un modulo n au haché SHA-256 ($H(msg) \pmod n$). Si n est plus petit que l'espace de hachage (2^{256}), il y a des collisions inévitables. *Standard moderne* : Le standard **PKCS#1 v2.1** exige l'utilisation du schéma **PSS** (Probabilistic Signature Scheme) pour les signatures RSA.

5. Conclusion

Ce projet constitue une **maquette pédagogique réussie** du cryptosystème RSA. Il permet de visualiser de manière concrète et interactive le lien entre l'arithmétique modulaire et les

protocoles de sécurité de l'information (confidentialité et authentification).

Les limites soulevées précédemment ne discréditent pas le travail accompli, mais illustrent parfaitement la transition entre la *théorie mathématique pure* et l'*ingénierie cryptographique appliquée*. Ce programme offre donc une excellente base de réflexion pour aborder, dans un cursus universitaire, les normes complexes de la cryptographie moderne (RFC 8017 pour PKCS#1).